

Univ.-Prof. Dr. Daniel Große
DI Simon Reitinger & Katharina Ruep, M.Sc.

11. Juni 2026

Übung 8

zur Vorlesung Rechnerarchitektur

Abgabe bis **Donnerstag 18 Juni, 2026 10:15** via EPICS: <https://ep.ics.jku.at>.

Aufgabe 1 – Leistungsbewertung

(9 Punkte)

Wir analysieren die drei RISC-V Prozessoren aus der Vorlesung:

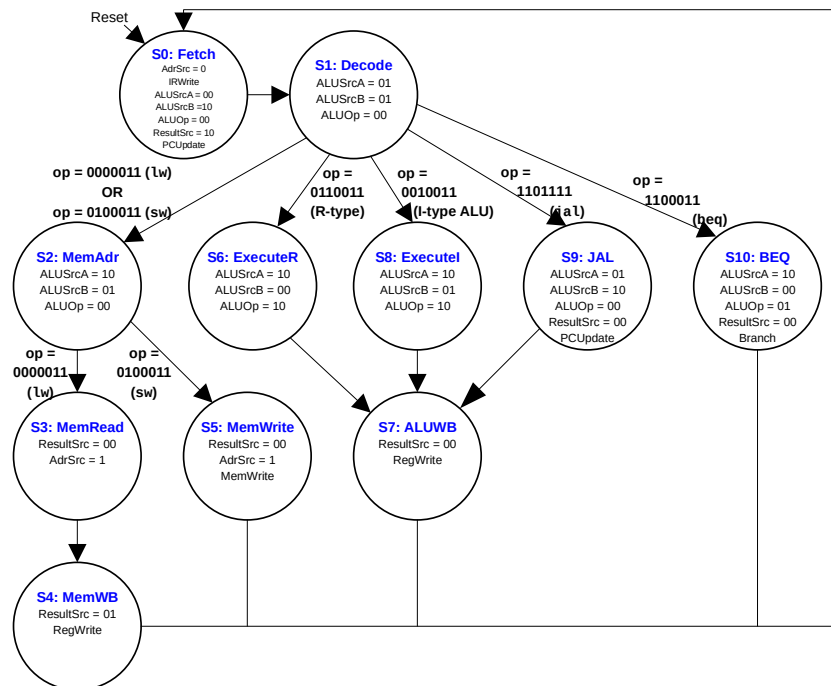
- **Single-cycle** Prozessor P_1 mit einer Taktfrequenz von 1 GHz.
 - **Pipelined** Prozessor P_2 mit 5-stufiger Pipeline und einer Taktfrequenz von 2 GHz.
 - Jumps und Branches werden in der Stufe *execute* ausgeführt.
 - Bei **Kontrollkonflikten** werden Befehle aus der Pipeline gelöscht.
 - Aufgrund von **Datenabhängigkeiten** muss nach jeder **achten** arithmetischen Operation (R-Typ & I-Typ) ein Takt angehalten werden (wie NOP).
 - Beachte für P_2 auch, dass die Pipeline initial befüllt werden muss bevor mehrere Befehle parallel ausgeführt werden können.
 - **Multicycle** Prozessor P_3 mit einer Taktfrequenz von 2,5 GHz.
 - Die Zyklen je Instruktion sollen aus der FSM in Figure 1 extrahiert werden.
- a) Berechne **CPI-Wert** und **Ausführungszeit** für jeden der drei Prozessoren für ein Programm mit **2,0 Milliarden Instruktionen** ($2 \cdot 10^9$) und dem in Table 1 gegebenen Instruktions-Mix. Gib den Rechenweg mit an.
- b) Bestimme den **MIPS-Wert** der Prozessoren P_1 , P_2 und P_3 . Gib den Rechenweg mit an.

Tabelle 1: RISC-V Instruktions-Mix

Befehl	rel. Häufigkeit
R-Typ & I-Typ	35 %
lw	25 %
sw	15 %
beq ¹	15 %
jal	10 %

¹ Bei 40 % der beq Befehle wird der Sprung getätigt.

Abbildung 1: Multicycle RISC-V FSM



Aufgabe 2 – Direct-mapped Cache

(6 Punkte)

In einem Rechner, der **lesend und schreibend** auf einen Speicher zugreift und **keine Byte-Adressierung** erlaubt, soll ein *direct-mapped Cache* mit **4 Cacheblöcken** und dem **Write-Back-Verfahren** verwendet werden. In einem auf dem Rechner laufenden Programm wird nacheinander auf folgende Adressen zugegriffen:

lw 0x2, lw 0xC, sw 0xC, sw 0xA, lw 0x1, lw 0xC, lw 0x8, sw 0x3,
lw 0x8, sw 0x8

Stelle den Zustand des Caches nach den jeweiligen Speicherzugriffen tabellarisch dar (Tag und Dirty-Bit). Kennzeichne in Zeile *H*, ob ein Hit oder Miss erfolgt und in Zeile *W*, wann ein Write-Back passiert.

	lw 0x2		lw 0xC		sw 0xC		sw 0xA		lw 0x1		lw 0xC		lw 0x8		sw 0x3		lw 0x8		sw 0x8	
	D	Tag	D	Tag	D	Tag	D	Tag	D	Tag	D	Tag	D	Tag	D	Tag	D	Tag	D	Tag
00																				
01																				
10																				
11																				
<i>H</i>																				
<i>W</i>																				

Aufgabe 3 – Assoziativer Cache

(9 Punkte)

In einem Rechner, der nur lesend auf einen Speicher zugreift und keine Byte-Adressierung erlaubt, soll ein *assoziativer Cache* mit 4 Cacheblöcken verwendet werden. In einem auf dem Rechner laufenden Programm werden nacheinander folgende Adressen gelesen:

0x7, 0x8, 0xA, 0x7, 0xB, 0xB, 0xD, 0x7, 0x3, 0x5, 0x9, 0x5, 0x7, 0x8, 0x9

- a) Fertige eine Tabelle an, welche den Zustand des Caches nach jeder geladenen Adresse zeigt, wenn FIFO (first in first out) als Verdrängungsstrategie verwendet wird. Zu wie vielen Cache-Misses kommt es?
- b) Fertige eine Tabelle an, welche den Zustand des Caches nach jeder geladenen Adresse zeigt, wenn LRU (least recently used) als Verdrängungsstrategie verwendet wird. Zu wie vielen Cache-Misses kommt es?
- c) Nimm an, dass der Zugriff auf den Hauptspeicher 250 ns dauert. Es stehen zwei Arten von Caches zur Verfügung: (i) Ein FIFO-Cache mit 20 ns Zugriffszeit und einer Trefferrate von 90 % und (ii) ein LRU-Cache mit 95 % Trefferrate. Wie lange darf die Zugriffszeit des LRU-Caches maximal sein, sodass der LRU-Cache im Durchschnitt schneller ist?

Aufgabe 4 – (Bonus) Direct-mapped Cache

(3 Punkte)

Es soll ein *direct-mapped Cache* mit einer Kapazität von 4 MiB (= 4096 KiB) realisiert werden. In einem Cacheblock sind **4 Datenwörter** abgespeichert, wobei ein Datenwort **32 Bit** breit ist. Bei Schreibzugriffen wird das **Write-Back** Verfahren angewendet. Der zugehörige Prozessor verwendet **32-Bit Adressen** und unterstützt eine **byteweise** Adressierung. Wie viele Speicherbits (1-Bit Flip-Flops) werden benötigt, um den Cache zu realisieren. Führe Nebenrechnungen an.